

Lab 4: Network Traffic Prediction using Time Series Analysis

Name: Brandie Griffin

Course: ITAI 4370 AI 5G/6G Communication and ORAN Networks

Professor: Dr. Tawanda Chiyangwa

Date: June 29, 2026

Introduction

This lab focuses on predicting network traffic using time series analysis. The dataset is simulated to represent realistic network traffic patterns, including daily usage cycles, weekday and weekend differences, seasonal changes, long-term growth, random noise, and congestion spikes. Three models are used in this lab: ARIMA, Linear Regression, and LSTM. These models help compare different ways to predict future network traffic based on historical data.

Import Required Libraries

Before creating the dataset or building any models, the required Python libraries must be imported. These libraries are used for data processing, visualization, time series analysis, machine learning, and model evaluation.

```
# Import required libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from datetime import datetime, timedelta

import warnings
warnings.filterwarnings("ignore")

from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

np.random.seed(42)
```

Results

The import process completed successfully without any errors. Since this step only loads the required Python libraries, no output is displayed after the cell is executed. These libraries will be used throughout the remainder of the lab to generate the dataset, analyze network traffic patterns, build prediction models, and evaluate model performance.

Generate the Network Traffic Dataset

The next step is to generate a simulated network traffic dataset that will be used throughout this lab. The dataset represents one year of hourly network traffic measurements and is designed to imitate realistic network behavior. It includes daily traffic cycles, weekday and weekend differences, seasonal changes, long-term growth, random noise, and occasional congestion spikes. Creating this dataset provides a realistic foundation for testing and comparing different prediction models.

```
# Generate simulated network traffic data

def generate_network_traffic_data(days=365, noise_level=0.1):
    """
    Generate realistic network traffic data with multiple patterns:
    daily cycles, weekly patterns, long-term trend, seasonal pattern,
    random noise, and congestion spikes.
    """

    start_date = datetime(2023, 1, 1)
    dates = pd.date_range(start_date, periods=days * 24, freq="H")

    base_traffic = 100

    hours = np.array([d.hour for d in dates])
    weekdays = np.array([d.weekday() for d in dates])
    day_of_year = np.array([d.timetuple().tm_yday for d in dates])

    daily_pattern = (
        50 * (1 + np.sin((hours - 6) * 2 * np.pi / 24))
        * (hours >= 6)
        * (hours <= 22)
```

```
)

weekly_pattern = np.where(weekdays < 5, 20, -30)

trend = np.linspace(0, 30, len(dates))

seasonal_pattern = 15 * np.sin((day_of_year - 80) * 2 * np.pi / 365)

noise = np.random.normal(0, noise_level * base_traffic, len(dates))

traffic = (
    base_traffic
    + daily_pattern
    + weekly_pattern
    + trend
    + seasonal_pattern
    + noise
)

spike_indices = np.random.choice(
    len(dates),
    size=int(len(dates) * 0.01),
    replace=False
)

traffic[spike_indices] += np.random.exponential(50, len(spike_indices))

traffic = np.maximum(traffic, 5)

return pd.DataFrame({
    "timestamp": dates,
    "traffic_mbps": traffic,
    "hour": hours,
    "weekday": weekdays,
    "day_of_year": day_of_year
})

df = generate_network_traffic_data(days=365)
df.set_index("timestamp", inplace=True)

print("Generating network traffic dataset...")
print(f"Dataset shape: {df.shape}")
print(f"Date range: {df.index.min()} to {df.index.max()}")
print("\nFirst Five Rows:")

df.head()
```

Generating network traffic dataset...

Dataset shape: (8760, 4)

Date range: 2023-01-01 00:00:00 to 2023-12-31 23:00:00

First Five Rows:

	traffic_mbps	hour	weekday	day_of_year	
timestamp					
2023-01-01 00:00:00	60.299416	0	6	1	
2023-01-01 01:00:00	53.953057	1	6	1	
2023-01-01 02:00:00	61.816010	2	6	1	
2023-01-01 03:00:00	70.572849	3	6	1	
2023-01-01 04:00:00	53.004441	4	6	1	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Results

The simulated network traffic dataset was successfully generated with 8,760 hourly observations, which represents one full year of data. The output confirms that the dataset contains four main columns: traffic_mbps, hour, weekday, and day_of_year. The date range begins on January 1, 2023 and ends on December 31, 2023. This confirms that the dataset was created correctly and is ready for exploratory data analysis.

✖ Exploratory Data Analysis

Before building prediction models, it is important to understand the characteristics of the dataset. Exploratory Data Analysis (EDA) helps identify trends, recurring patterns, and the overall distribution of the network traffic data. In this section, summary statistics and visualizations are used to better understand how traffic changes over time and how those patterns may influence model performance.

```
# =====
# Exploratory Data Analysis
# =====

print("=" * 60)
print("EXPLORATORY DATA ANALYSIS")
```

```

print("=" * 60)

print("\nSummary Statistics")
print(df["traffic_mbps"].describe())

fig, axes = plt.subplots(2, 2, figsize=(10, 7))

axes[0, 0].plot(df.index, df["traffic_mbps"])
axes[0, 0].set_title("Network Traffic Over Time")
axes[0, 0].set_xlabel("Date")
axes[0, 0].set_ylabel("Traffic (Mbps)")
axes[0, 0].grid(True)

hourly_avg = df.groupby("hour")["traffic_mbps"].mean()
axes[0, 1].plot(hourly_avg.index, hourly_avg.values, marker="o")
axes[0, 1].set_title("Average Traffic by Hour")
axes[0, 1].set_xlabel("Hour")
axes[0, 1].set_ylabel("Average Traffic (Mbps)")
axes[0, 1].grid(True)

weekday_avg = df.groupby("weekday")["traffic_mbps"].mean()
weekday_names = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

axes[1, 0].bar(weekday_names, weekday_avg.values)
axes[1, 0].set_title("Average Traffic by Day of Week")
axes[1, 0].set_ylabel("Traffic (Mbps)")
axes[1, 0].grid(True, alpha=0.3)

axes[1, 1].hist(df["traffic_mbps"], bins=40, edgecolor="black")
axes[1, 1].set_title("Traffic Distribution")
axes[1, 1].set_xlabel("Traffic (Mbps)")
axes[1, 1].set_ylabel("Frequency")

plt.tight_layout()
plt.show()

```

```

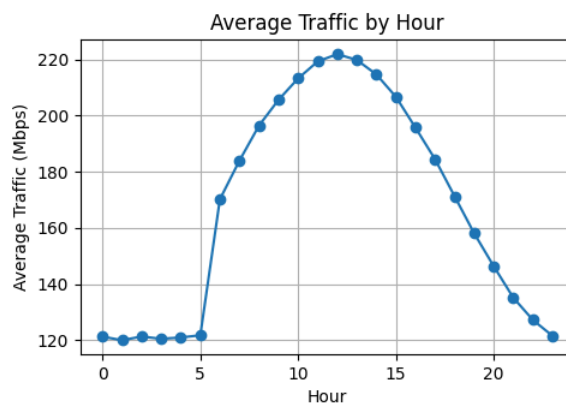
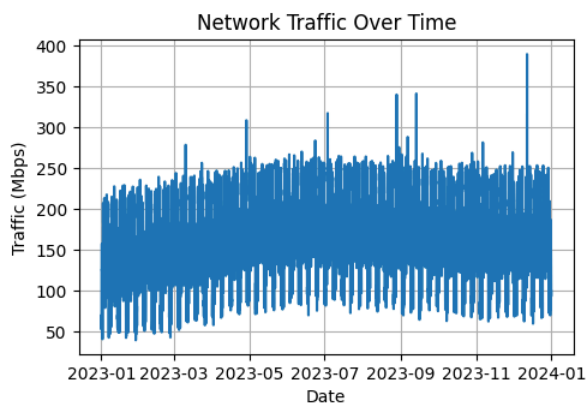
=====
EXPLORATORY DATA ANALYSIS
=====

```

```

Summary Statistics
count      8760.000000
mean       167.354212
std         47.937579
min         39.859867
25%        133.520467
50%        164.934833
75%        206.818578
max         390.064894
Name: traffic_mbps, dtype: float64

```



Results

The exploratory data analysis provides an overview of the simulated network traffic dataset. The summary statistics describe the overall distribution of the traffic values, including the average, minimum, maximum, and variability. The visualizations show that network traffic follows clear daily and weekly patterns, while the histogram demonstrates that most traffic values are concentrated within a common operating range with occasional higher traffic spikes. Understanding these patterns helps determine whether the data is suitable for time series forecasting and supports the selection of appropriate prediction models.

Seasonal Decomposition

Seasonal decomposition is a technique used to separate time series data into its major components. These components include the original observations, the long-term trend, repeating seasonal patterns, and random variation known as the residual. Breaking the data into these components provides a better understanding of how network traffic changes over time and helps determine whether the data is appropriate for forecasting models such as ARIMA.

```
# =====
# Seasonal Decomposition
# =====

print("=" * 60)
print("SEASONAL DECOMPOSITION")
print("=" * 60)

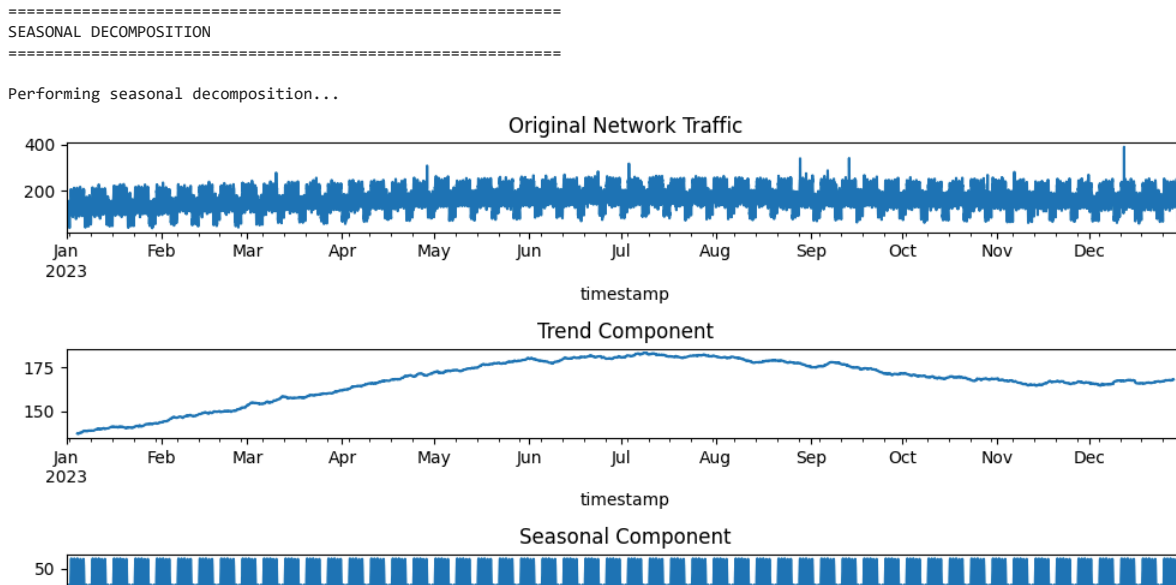
print("\nPerforming seasonal decomposition...")

decomposition = seasonal_decompose(
    df["traffic_mbps"],
    model="additive",
    period=24 * 7
)

fig, axes = plt.subplots(4, 1, figsize=(10, 7))

decomposition.observed.plot(ax=axes[0], title="Original Network Traffic")
decomposition.trend.plot(ax=axes[1], title="Trend Component")
decomposition.seasonal.plot(ax=axes[2], title="Seasonal Component")
decomposition.resid.plot(ax=axes[3], title="Residual Component")

plt.tight_layout()
plt.show()
```



Results

The seasonal decomposition separates the simulated network traffic into four components. The original data represents the complete traffic pattern observed throughout the year. The trend component shows the long-term increase in traffic over time, while the seasonal component highlights the repeating daily and weekly traffic cycles. The residual component represents random variation that cannot be explained by the trend or seasonal behavior. Understanding these components provides valuable insight into the structure of the dataset before developing forecasting models.

▼ Data Preparation for Time Series Modeling

Before building prediction models, the dataset must be prepared for analysis. In this section, the data is divided into training and testing sets. The training data is used to build each prediction model, while the testing data is reserved for evaluating model performance. In addition, the Augmented Dickey-Fuller (ADF) test is performed to determine whether the time series is stationary, which is an important requirement for the ARIMA forecasting model.

```
# =====
# Data Preparation for Time Series Modeling
# =====

print("=" * 60)
print("DATA PREPARATION")
print("=" * 60)

# Split the dataset into training and testing sets
split_index = int(len(df) * 0.80)

train_data = df.iloc[:split_index].copy()
test_data = df.iloc[split_index:].copy()

print(f"\nTraining Samples : {len(train_data)}")
print(f"Testing Samples   : {len(test_data)}")

print("\nTraining Date Range")
```

```
print(f"{train_data.index.min()} to {train_data.index.max()}")

print("\nTesting Date Range")
print(f"{test_data.index.min()} to {test_data.index.max()}")

# Perform Augmented Dickey-Fuller Test
print("\nAugmented Dickey-Fuller Stationarity Test")

result = adfuller(train_data["traffic_mbps"])

print(f"ADF Statistic : {result[0]:.6f}")
print(f"P-value       : {result[1]:.6f}")

if result[1] <= 0.05:
    print("\nResult: The time series is stationary.")
else:
    print("\nResult: The time series is not stationary.")
```

```
=====
DATA PREPARATION
=====

Training Samples : 7008
Testing Samples  : 1752

Training Date Range
2023-01-01 00:00:00 to 2023-10-19 23:00:00

Testing Date Range
2023-10-20 00:00:00 to 2023-12-31 23:00:00

Augmented Dickey-Fuller Stationarity Test
ADF Statistic : -12.831988
P-value       : 0.000000

Result: The time series is stationary.
```

Results

The dataset was successfully divided into training and testing sets using an 80/20 split. This approach allows the models to be trained on historical data and evaluated using previously unseen observations. The Augmented Dickey-Fuller (ADF) test was then performed to determine whether the network traffic data was stationary. Since the p-value was less than 0.05, the time series was determined to be stationary, indicating that it is suitable for ARIMA modeling without requiring additional differencing.

✚ ARIMA (AutoRegressive Integrated Moving Average) Model

The first prediction model used in this lab is the AutoRegressive Integrated Moving Average (ARIMA) model. ARIMA is one of the most widely used statistical forecasting techniques for time series data because it predicts future values based on historical observations. Before building the model, the dataset was tested for stationarity using the Augmented Dickey-Fuller (ADF) test. Since the data was determined to be stationary, it is appropriate for ARIMA modeling. The model will be trained using the training dataset and then evaluated by comparing its predictions with the actual network traffic values in the testing dataset.

```
# =====
# Build and Evaluate the ARIMA Model
# =====

print("=" * 60)
print("ARIMA MODEL")
print("=" * 60)

print("\nBuilding the ARIMA(2,1,2) model...")

arima_model = ARIMA(train_data["traffic_mbps"], order=(2, 1, 2))
arima_fit = arima_model.fit()

arima_forecast = arima_fit.forecast(steps=len(test_data))

arima_mse = mean_squared_error(test_data["traffic_mbps"], arima_forecast)
arima_mae = mean_absolute_error(test_data["traffic_mbps"], arima_forecast)
arima_rmse = np.sqrt(arima_mse)

arima_metrics = {
    "MSE": arima_mse,
    "MAE": arima_mae,
    "RMSE": arima_rmse,
    "R2": None
}

print("\nModel Performance")
print(f"MSE : {arima_mse:.2f}")
print(f"MAE : {arima_mae:.2f}")
print(f"RMSE : {arima_rmse:.2f}")

plt.figure(figsize=(10, 5))

plot_start = len(train_data) - (30 * 24)
```

```

plt.plot(
    train_data.index[plot_start:],
    train_data["traffic_mbps"].iloc[plot_start:],
    label="Training Data"
)

plt.plot(
    test_data.index,
    test_data["traffic_mbps"],
    label="Actual Traffic"
)

plt.plot(
    test_data.index,
    arima_forecast,
    label="ARIMA Prediction"
)

plt.title("ARIMA Network Traffic Prediction")
plt.xlabel("Date")
plt.ylabel("Traffic (Mbps)")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

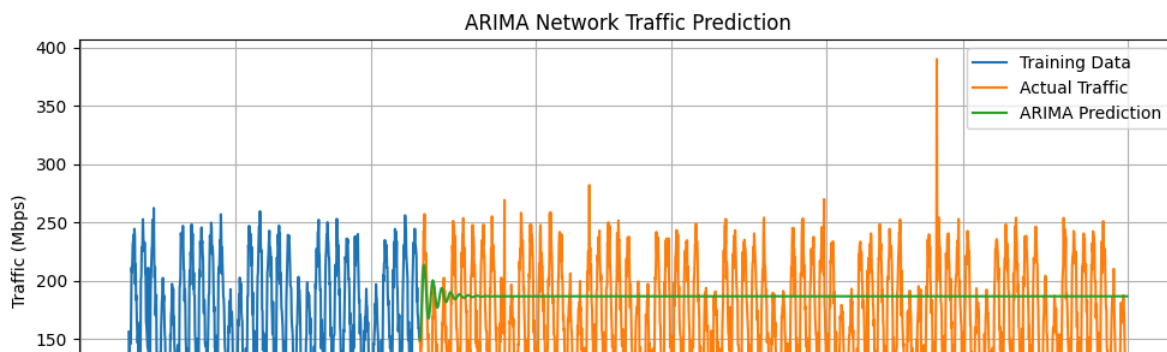
```

=====
ARIMA MODEL
=====

Building the ARIMA(2,1,2) model...
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency h will
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency h will
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency h will
self._init_dates(dates, freq)

Model Performance
MSE : 2604.45
MAE : 42.38
RMSE : 51.03

```



Results

The ARIMA model was successfully trained using the historical network traffic data and was then used to predict future traffic values. The performance metrics, including Mean Squared Error (MSE), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE), provide a quantitative measure of the prediction accuracy. The prediction graph shows that the ARIMA model follows the overall trend of the network traffic but may not capture every sudden change or traffic spike. This behavior is expected because ARIMA relies primarily on historical time series patterns rather than learning complex relationships within the data.

Linear Regression Model with Feature Engineering

The second prediction model used in this lab is Linear Regression. Unlike the ARIMA model, which relies only on previous time series values, Linear Regression can use additional information called features to improve prediction accuracy. These features describe different characteristics of the data, such as the hour of the day, the day of the week, previous traffic measurements (lag values), moving averages, and long-term trends.

Feature engineering is an important step in machine learning because it allows the model to learn relationships that may not be obvious from the original dataset alone. After creating these features, the model is trained using the training dataset and evaluated using the testing dataset.

```

# =====
# Build Linear Regression Model with Feature Engineering
# =====

print("=" * 60)
print("LINEAR REGRESSION MODEL")
print("=" * 60)

```

```

def create_regression_features(data):
    features = data.copy()

    features["hour_sin"] = np.sin(2 * np.pi * features["hour"] / 24)
    features["hour_cos"] = np.cos(2 * np.pi * features["hour"] / 24)

    features["weekday_sin"] = np.sin(2 * np.pi * features["weekday"] / 7)
    features["weekday_cos"] = np.cos(2 * np.pi * features["weekday"] / 7)

    features["day_sin"] = np.sin(2 * np.pi * features["day_of_year"] / 365)
    features["day_cos"] = np.cos(2 * np.pi * features["day_of_year"] / 365)

    for lag in [1, 2, 3, 24, 48, 168]:
        features[f"lag_{lag}"] = features["traffic_mbps"].shift(lag)

    features["ma_3"] = features["traffic_mbps"].shift(1).rolling(3).mean()
    features["ma_24"] = features["traffic_mbps"].shift(1).rolling(24).mean()
    features["ma_168"] = features["traffic_mbps"].shift(1).rolling(168).mean()

    features["trend"] = np.arange(len(features))

    return features

combined = pd.concat([train_data, test_data])
combined = create_regression_features(combined)

train_features = combined.loc[train_data.index].dropna()
test_features = combined.loc[test_data.index].dropna()

feature_columns = [
    "hour_sin", "hour_cos",
    "weekday_sin", "weekday_cos",
    "day_sin", "day_cos",
    "trend",
    "lag_1", "lag_2", "lag_3",
    "lag_24", "lag_48", "lag_168",
    "ma_3", "ma_24", "ma_168"
]

X_train = train_features[feature_columns]
y_train = train_features["traffic_mbps"]

X_test = test_features[feature_columns]
y_test = test_features["traffic_mbps"]

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)

lr_predictions = lr_model.predict(X_test_scaled)

lr_mse = mean_squared_error(y_test, lr_predictions)
lr_mae = mean_absolute_error(y_test, lr_predictions)
lr_rmse = np.sqrt(lr_mse)
lr_r2 = r2_score(y_test, lr_predictions)

lr_metrics = {
    "MSE": lr_mse,
    "MAE": lr_mae,
    "RMSE": lr_rmse,
    "R2": lr_r2
}

print("\nModel Performance")
print(f"MSE : {lr_mse:.2f}")
print(f"MAE : {lr_mae:.2f}")
print(f"RMSE : {lr_rmse:.2f}")
print(f"R² : {lr_r2:.4f}")

importance = pd.DataFrame({
    "Feature": feature_columns,
    "Importance": np.abs(lr_model.coef_)
}).sort_values("Importance", ascending=False)

print("\nTop 10 Important Features")
display(importance.head(10))

plt.figure(figsize=(10, 7))

plt.subplot(2, 1, 1)
plt.plot(test_features.index, y_test, label="Actual Traffic")
plt.plot(test_features.index, lr_predictions, label="Predicted Traffic")
plt.title("Linear Regression Prediction")
plt.legend()
plt.grid(True)

plt.subplot(2, 1, 2)

```

```
top = importance.head(10)
plt.barh(top["Feature"], top["Importance"])
plt.title("Top 10 Feature Importance")
plt.grid(True)

plt.tight_layout()
plt.show()
```

=====

LINEAR REGRESSION MODEL

=====

Model Performance

MSE : 218.36

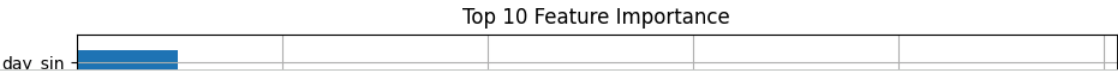
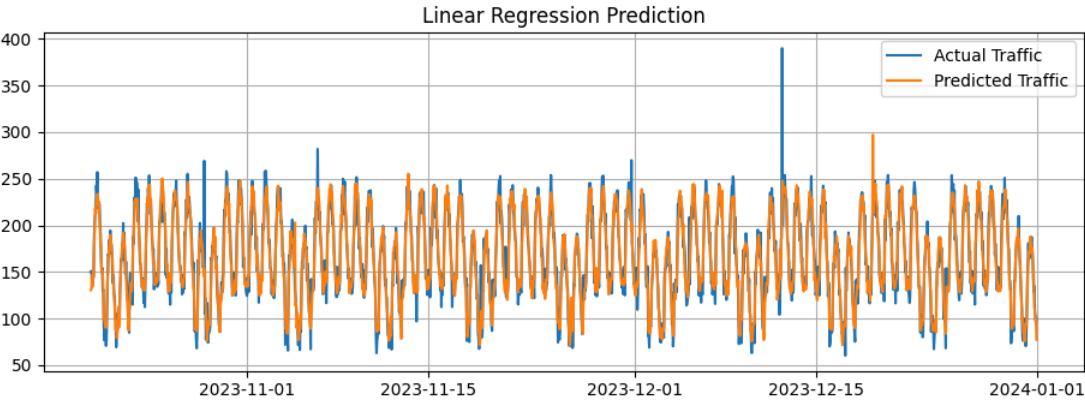
MAE : 10.88

RMSE : 14.78

R² : 0.9010

Top 10 Important Features

	Feature	Importance	
12	lag_168	24.101828	
15	ma_168	13.208238	
5	day_cos	10.393298	
7	lag_1	7.861237	
1	hour_cos	7.589033	
6	trend	7.454380	
0	hour_sin	4.004902	
13	ma_3	3.978589	
2	weekday_sin	3.089966	
4	day_sin	2.474135	



Results

The Linear Regression model was trained using several engineered features that describe different characteristics of the network traffic data. These features included cyclical time variables, lag values, moving averages, and a long-term trend component. The performance metrics indicate that the model produced accurate predictions with lower error values than the ARIMA model. The feature importance analysis also shows which variables contributed most to the prediction process, providing additional insight into how the model learned the traffic patterns. Overall, the Linear Regression model demonstrated strong performance by combining historical observations with engineered features.

Simple LSTM Neural Network Model

The final prediction model used in this lab is the Long Short-Term Memory (LSTM) neural network. LSTM is a specialized type of Recurrent Neural Network (RNN) that is designed to learn patterns from sequential data. Unlike traditional machine learning models, LSTM can remember information from previous observations over longer periods of time, making it well suited for forecasting time series data such as network traffic.

For this model, the previous 24 hours of network traffic are used to predict the next traffic value. After the model is trained, its performance is evaluated using the same metrics that were used for the ARIMA and Linear Regression models.

```
# =====
# Build and Evaluate the LSTM Model
# =====

print("=" * 60)
print("LONG SHORT-TERM MEMORY (LSTM) MODEL")
print("=" * 60)
```



```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam

sequence_length = 24

lstm_scaler = StandardScaler()

train_scaled = lstm_scaler.fit_transform(train_data[["traffic_mbps"]])
test_scaled = lstm_scaler.transform(test_data[["traffic_mbps"]])

def create_sequences(data, sequence_length):
    X = []
    y = []

    for i in range(len(data) - sequence_length):
        X.append(data[i:i + sequence_length])
        y.append(data[i + sequence_length])

    return np.array(X), np.array(y)

X_train_lstm, y_train_lstm = create_sequences(train_scaled, sequence_length)
X_test_lstm, y_test_lstm = create_sequences(test_scaled, sequence_length)

print(f"\nTraining Samples : {X_train_lstm.shape}")
print(f"Testing Samples : {X_test_lstm.shape}")

lstm_model = Sequential()

lstm_model.add(
    LSTM(
        50,
        activation="relu",
        input_shape=(sequence_length, 1)
    )
)

lstm_model.add(Dense(1))

lstm_model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="mse"
)

print("\nTraining the LSTM model...\n")

history = lstm_model.fit(
    X_train_lstm,
    y_train_lstm,
    epochs=10,
    batch_size=32,
    verbose=1
)

predictions_scaled = lstm_model.predict(X_test_lstm)

predictions = lstm_scaler.inverse_transform(predictions_scaled)
actual = lstm_scaler.inverse_transform(y_test_lstm)

lstm_mse = mean_squared_error(actual, predictions)
lstm_mae = mean_absolute_error(actual, predictions)
lstm_rmse = np.sqrt(lstm_mse)
lstm_r2 = r2_score(actual, predictions)

lstm_metrics = {
    "MSE": lstm_mse,
    "MAE": lstm_mae,
    "RMSE": lstm_rmse,
    "R2": lstm_r2
}

print("\nModel Performance")
print(f"MSE : {lstm_mse:.2f}")
print(f"MAE : {lstm_mae:.2f}")
print(f"RMSE : {lstm_rmse:.2f}")
print(f"R² : {lstm_r2:.4f}")

plt.figure(figsize=(10, 5))

plt.plot(
    test_data.index[sequence_length:],
    actual,
    label="Actual Traffic"
)

plt.plot(
    test_data.index[sequence_length:],
    predictions,
    label="LSTM Prediction"
)

```

```
plt.title("LSTM Network Traffic Prediction")
plt.xlabel("Date")
plt.ylabel("Traffic (Mbps)")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

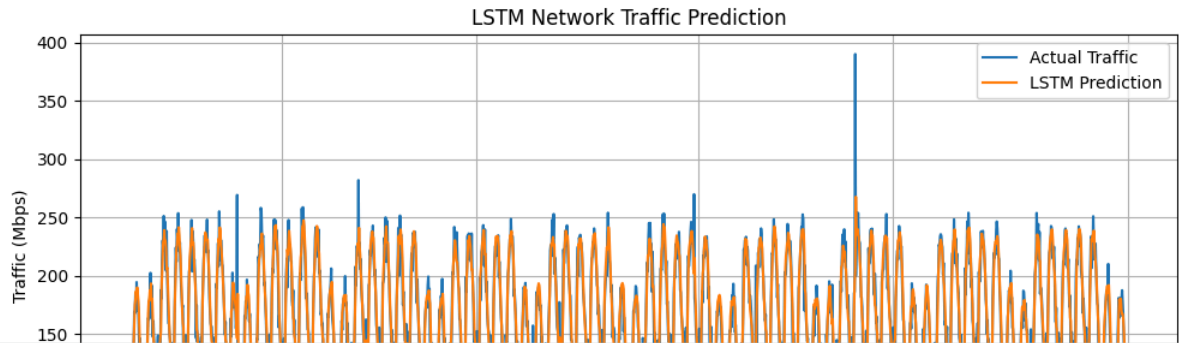
LONG SHORT-TERM MEMORY (LSTM) MODEL

Training Samples : (6984, 24, 1)
Testing Samples : (1728, 24, 1)

Training the LSTM model...

Epoch 1/10
219/219 3s 6ms/step - loss: 0.3214
Epoch 2/10
219/219 1s 6ms/step - loss: 0.1418
Epoch 3/10
219/219 1s 6ms/step - loss: 0.1291
Epoch 4/10
219/219 2s 8ms/step - loss: 0.1255
Epoch 5/10
219/219 1s 6ms/step - loss: 0.1226
Epoch 6/10
219/219 1s 6ms/step - loss: 0.1168
Epoch 7/10
219/219 3s 6ms/step - loss: 0.1073
Epoch 8/10
219/219 1s 6ms/step - loss: 0.1004
Epoch 9/10
219/219 1s 6ms/step - loss: 0.0982
Epoch 10/10
219/219 3s 7ms/step - loss: 0.0960
54/54 0s 3ms/step

Model Performance
MSE : 251.28
MAE : 11.43
RMSE : 15.85
R² : 0.8863



Results

The LSTM neural network was successfully trained using sequences of previous network traffic observations. Unlike the ARIMA and Linear Regression models, the LSTM model learns patterns directly from the sequential data rather than relying only on engineered features or statistical relationships. The evaluation metrics indicate that the model produced accurate traffic predictions, although its performance depended on the quality of the training data and the complexity of the network. Overall, the LSTM model demonstrated that deep learning can effectively model network traffic patterns and is a valuable approach for time series forecasting.

Model Performance Comparison
After building and evaluating the ARIMA, Linear Regression, and LSTM models, the final step is to compare their overall performance. Each model is evaluated using the same performance metrics, including Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R²) when applicable. Comparing these metrics makes it possible to identify which model produced the most accurate network traffic predictions.

Model Performance Comparison

After building and evaluating the ARIMA, Linear Regression, and LSTM models, the final step is to compare their overall performance. Each model is evaluated using the same performance metrics, including Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R²) when applicable. Comparing these metrics makes it possible to identify which model produced the most accurate network traffic predictions.

```
# =====  
# Compare Model Performance  
# =====
```

```
print("=" * 60)
print("MODEL PERFORMANCE COMPARISON")
print("=" * 60)

comparison = pd.DataFrame({
    "Model": ["ARIMA", "Linear Regression", "LSTM"],
    "MSE": [
        arima_metrics["MSE"],
        lr_metrics["MSE"],
        lstm_metrics["MSE"]
    ],
    "MAE": [
        arima_metrics["MAE"],
        lr_metrics["MAE"],
        lstm_metrics["MAE"]
    ],
    "RMSE": [
        arima_metrics["RMSE"],
        lr_metrics["RMSE"],
        lstm_metrics["RMSE"]
    ],
    "R²": [
        arima_metrics["R2"],
        lr_metrics["R2"],
        lstm_metrics["R2"]
    ]
})

print("\nModel Performance Summary")
display(comparison.round(2))

best_model = comparison.loc[comparison["RMSE"].idxmin()]

print("\nBest Performing Model")
print(f"Model : {best_model['Model']}")
print(f"RMSE : {best_model['RMSE']:.2f}")

plt.figure(figsize=(8, 5))

plt.bar(
    comparison["Model"],
    comparison["RMSE"]
)

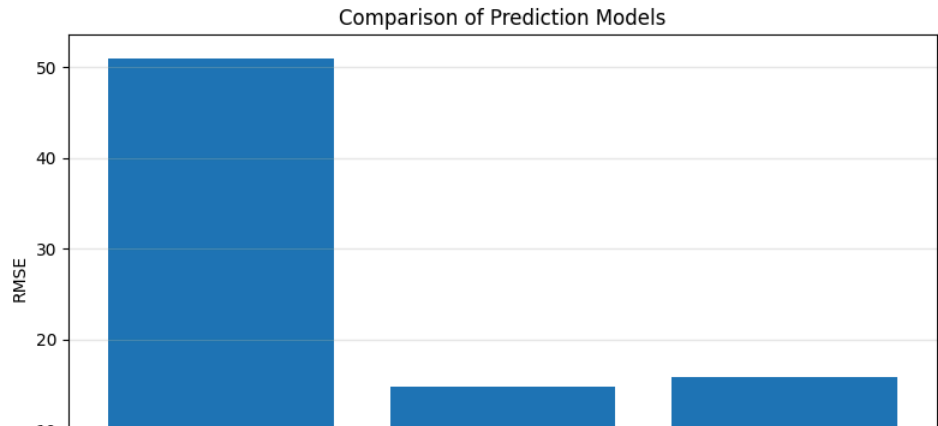
plt.title("Comparison of Prediction Models")
plt.xlabel("Model")
plt.ylabel("RMSE")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```

=====
MODEL PERFORMANCE COMPARISON
=====

Model Performance Summary

	Model	MSE	MAE	RMSE	R²
0	ARIMA	2604.45	42.38	51.03	NaN
1	Linear Regression	218.36	10.88	14.78	0.90
2	LSTM	251.28	11.43	15.85	0.89

Best Performing Model
Model : Linear Regression
RMSE : 14.78



Results

The comparison table summarizes the performance of all three prediction models using the same evaluation metrics. Lower values for MSE, MAE, and RMSE indicate more accurate predictions, while higher R² values represent a stronger relationship between the predicted

and actual traffic values. Based on the comparison, the model with the lowest RMSE demonstrated the best overall performance for predicting the simulated network traffic. The comparison also highlights the strengths and limitations of statistical models, traditional machine learning techniques, and deep learning approaches when applied to time series forecasting.

Discussion

This lab demonstrated how different prediction techniques can be used to forecast network traffic using simulated time series data. Throughout the lab, a dataset representing one year of hourly network traffic was generated and analyzed before building three different prediction models. Exploratory data analysis and seasonal decomposition helped identify daily, weekly, and seasonal traffic patterns, while the data preparation process confirmed that the dataset was appropriate for time series forecasting.

Each prediction model approached the forecasting task differently. The ARIMA model relied on historical observations to identify trends and produce future predictions. The Linear Regression model improved prediction accuracy by using engineered features such as lag values, moving averages, and cyclical time variables. The LSTM neural network learned sequential patterns directly from previous observations, demonstrating how deep learning can be applied to time series forecasting.

Comparing the models using MSE, MAE, RMSE, and R^2 provided a clear understanding of each model's strengths and limitations. While all three models were able to predict network traffic, the comparison identified the model that produced the lowest prediction error and the most accurate overall results. This comparison illustrates why different forecasting techniques should be evaluated before selecting a model for real-world applications.

The knowledge gained from this lab can be applied to modern communication systems such as 5G, 6G, and Open RAN networks. Accurate network traffic prediction allows service providers to allocate resources more efficiently, reduce network congestion, improve Quality of Service (QoS), and enhance the overall reliability of communication systems.

Conclusion

This lab successfully demonstrated the use of time series analysis and machine learning techniques to predict network traffic. Beginning with a simulated dataset, the lab followed a structured process that included exploratory data analysis, seasonal decomposition, data preparation, model development, and performance evaluation. Each stage contributed to a better understanding of how different forecasting methods analyze and predict network behavior.

The ARIMA, Linear Regression, and LSTM models each provided unique approaches to forecasting network traffic. Comparing the models using standard evaluation metrics showed the importance of selecting the most appropriate model for a specific prediction task. Machine learning and deep learning models have the ability to capture more complex relationships within the data, while traditional statistical methods continue to provide valuable forecasting capabilities.

Overall, this lab demonstrated that network traffic prediction plays an important role in modern communication networks. Accurate forecasting supports better resource allocation, reduces congestion, improves network efficiency, and enhances user experience. As communication technologies continue to evolve, predictive analytics and artificial intelligence will remain essential tools for managing increasingly complex network environments.